

README • PROJECT OVERVIEW



Ergalics Studio

A scientific computing workbench in the browser — interactive data exploration, GPU compute dispatch and a sandboxed plugin system, all running client-side with a Rust-powered WebAssembly core.

Live demo

GitHub SnowLeopard-io/ErgalicsStudio

MIT license

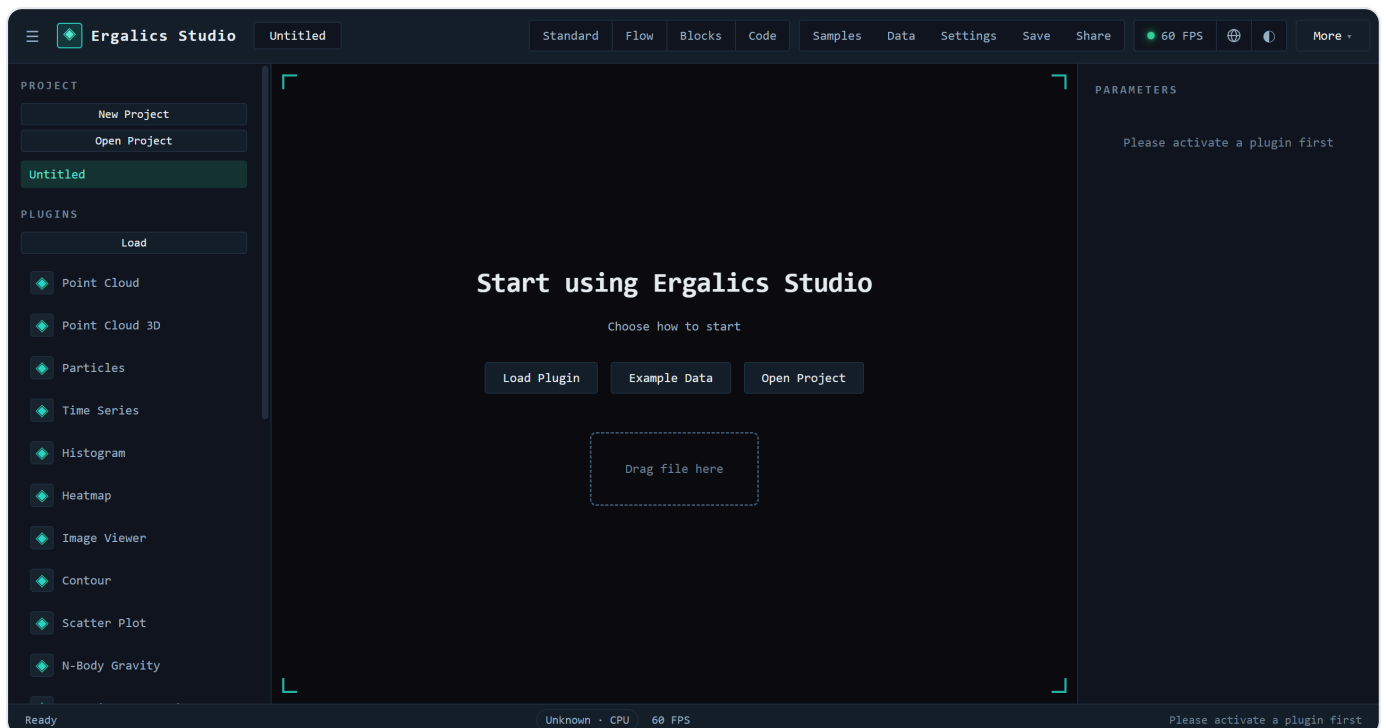
TypeScript 5.7

React 18

WebGPU • WGS

Rust to WASM

Pyodide (CPython)



Ergalics Studio — Standard mode (drag • see)

Table of Contents

- [Overview](#)
- [Features](#)
- [Architecture](#)

- [Tech Stack](#)
 - [Getting Started](#)
 - [Project Layout](#)
 - [Standard Mode](#)
 - [Flow Mode](#)
 - [Block Mode](#)
 - [Code Mode](#)
 - [Plugin System](#)
 - [GPU Compute & Native Core](#)
 - [Testing](#)
 - [Documentation](#)
 - [Roadmap](#)
 - [Contributing](#)
 - [License](#)
-

Overview

Ergalics Studio is a professional scientific-computing workstation that runs entirely in the browser. It combines a React + TypeScript frontend, a Rust core compiled to WebAssembly, a WebGPU compute pipeline, and a plugin architecture designed for third-party extensions.

The workbench exposes four modes for four kinds of users — see the section for each one below:

- **Standard** — drag a dataset onto a plugin, see the visualisation. The fastest path from "I have data" to "I see something".
- **Flow** — compose a visual dataflow pipeline from built-in blocks, run it topologically, inspect every node's output.
- **Block** — a Scratch-like block editor where a single "Run" hat block kicks off the program. Beginner-friendly, but fully scripted (variables, loops, conditionals, transforms, plots).
- **Code** — a Monaco Python editor backed by a Pyodide worker runtime, with the same `studio.*` API as block mode, a REPL console and a variable panel.

Ergalics Studio is under **active development** and already usable end to end: the core loop (project management, data loading, plugin registry, 2D/3D rendering, i18n, theming, performance monitoring, the Flow mode, the Block mode, and the Code mode with a Pyodide Python runtime) is functional and covered by tests. GPU acceleration spans Particles, N-Body, the LBM fluid, the wave equation, histogram, heatmap and point-cloud kernels; package signing for the plugin marketplace and the R runtime (webR) are the next milestones. Every module is kept deliberately small and testable so the codebase keeps scaling without a rewrite.

Status: **Active development** — usable today with four workbench modes, 40 built-in plugins (30 core + 10 fun), a sandboxed plugin system, a marketplace catalog, live GPU compute, an in-browser AI

training plugin, a statistics subsystem, scientific binary I/O (HDF5 / NetCDF / FITS / Zarr / Parquet), a publication-grade SVG/PDF plot engine, reproducibility support, and a Pyodide-powered Python code editor; package signing and the R runtime are next.

Features

Workbench

- Four-region layout: sidebar (projects / plugins), central viewport, right parameter panel, status bar with GPU/perf indicators. Project data files and settings live in the top bar (示例 | 数据 | 设置 | 保存 | 分享).
- Project lifecycle: create / open / save / autosave / share (`.clproj` format stored in IndexedDB).
- File routing: drag & drop any file; the host detects the format by magic number **and** extension (with optional WASM assist) and routes it to a matching plugin — with a picker dialog when multiple plugins match.

Rendering

- 2D canvas container shared by all 2D plugins (point clouds, particles, time series, histograms, heatmaps, image viewer, contour plots, scatter, bar charts, radar, network graphs, bubble charts, violin plots, sankey diagrams, box plots, parallel coordinates, error bands, treemaps, QQ plots, and the fractal/art toys).
- Host-managed **Three.js 3D scene** (`Scene3DHandle`): grid/axes/lights, orbit controls, resize handling, auto camera fit, and GPU-safe disposal. The 3D surface is created lazily — only for plugins that declare 3D capability (`renderToScene`) — and is **automatically hidden when a 2D plugin is active**, so a 3D coordinate system never bleeds into a 2D view.

Plugin system

- **40 built-in plugins** — 30 core/scientific plugins plus 10 fun & utility toys — covering the full API surface (2D canvas, Three.js scene, WGSL compute, buttons/toggles, sandboxing, in-browser model training).
- **Two-tier loading**: core plugins are auto-loaded at startup; fun/utility plugins declare `autoload: false` and are loaded on demand from the built-in panel or the marketplace tab, keeping the startup registry lean.
- **Marketplace catalog** (`src/plugins/marketplace.ts`) — every built-in plugin is surfaced with curated tags, popularity, and category filters (scientific / fun / utility); community "coming soon" submissions are listed as placeholders.
- `.cspkg` package loading (ZIP with `manifest.json` + entry + assets) with manifest validation (id format, entry path traversal guard, sandbox enum).
- **Real sandbox isolation** (§ 6.2): third-party entry code runs inside a Web Worker with a `postMessage` RPC bridge — no access to the host page's globals, DOM, or stores. Canvas rendering works via a transferred `OffscreenCanvas` ; a documented best-effort fallback exists when Workers are unavailable.

- Locale-aware parameter panels (range / select / number / checkbox / text / file / button / toggle).

Infrastructure

- i18n (zh-CN / en-US) with reactive locale switching.
- Dark/light theming via CSS variables.
- Performance monitor: FPS, frame time, GPU time, memory, data scale, with warning thresholds (§ 7.3).
- Error boundaries, fallbacks, and a banner/notification system.

Scientific computing subsystems (pure TypeScript, unit-tested)

- **Statistics kernel** (`src/core/stats/`) — descriptive statistics, special functions (incomplete gamma/beta, inverse CDFs), hypothesis tests (one/two-sample & paired t-tests, one-way ANOVA, Mann–Whitney U, chi-square independence), effect sizes (Cohen's d, Pearson/Spearman correlation), multiple-comparison corrections (Bonferroni, Benjamini–Hochberg) and two-sample power analysis.
- **Scientific binary I/O** (`src/core/io/`) — a single dispatcher routes dropped files to loaders for HDF5 (h5wasm), NetCDF (netcdfjs), FITS (fitsjs), Parquet (parquet-wasm) and Zarr (zarrita), turning each variable/dataset/HDU into project data files.
- **Publication-grade plot engine** (`src/core/plot/`) — a pure-TS SVG renderer with linear/log/temporal scales and nice ticks, plus SVG and PDF export for line, scatter, histogram and bar charts.
- **Reproducibility kernel** (`src/core/repro/`) — seeded RNG (mulberry32), stable hashing, run manifests (seed + version + input hashes + graph hash) and DAG-to-Python export for reruns.
- **Run-log export** (`src/core/logger.ts` + `download.ts`) — session logs can be exported from the workbench for bug reports.

Flow mode (visual dataflow pipeline)

- A second workbench mode next to Standard — toggle with the `Standard | Flow` switch in the top bar. Standard mode is *load data → see it*; Flow mode is *compose a visual pipeline → run it → see every node's output*.
- 37 built-in blocks organised by category: data sources, transforms, filters, math, statistics (including t-tests, ANOVA, Mann–Whitney, chi-square, correlation, effect sizes and multiple-comparison corrections), plotting and visualizations. Control-flow blocks (if/else, repeat, parallel) are deliberately deferred — the `region` seam on `BlockInstance` is in place so they slot in later as an extension, not a refactor.
- **Compiler is a pure function**: structural validation (ports / required inputs / type compatibility), cycle detection, and Kahn-style topological sort. Errors come back as structured `diagnostics` so the canvas can paint red edges and an inline diagnostic strip without ever throwing.
- **Executor with incremental caching** at per-node granularity plus a dirty-propagation invalidation pass — change a single block's parameter and only it and its downstream re-execute.
- **Result preview** with one click: `RenderedView` outputs go through the existing plugin renderer (scatter, histogram, ...); `DataTable` outputs render as a read-only table (so `stats.summary` / `stats.histogram` bins actually show something); `Scalar` outputs render inline. A chip switcher picks which node to inspect when a pipeline has more than one output.

- **Reactive parameter editor** bound to the selected node, with a two-way link to the canvas — the node card shows the live `key: value` summary, so you always see what the canvas is actually running.
- **Block metadata is localized** (`nameI18n` / `descriptionI18n`) and the palette, node card, and parameter panel all resolve via `src/blocks/i18n.ts`, so adding a locale is a data-only change.
- **Sample pipelines live as .clproj files** under `examples/projects/` (`block-01-signal-analysis.clproj`, ...). They are normal projects — loadable through the standard project picker — and discovered at build time via `import.meta.glob`. Adding a new sample is dropping a file plus an entry in `SAMPLE_META`.
- The whole graph persists into the project's `blockGraph` and rehydrates on open, sharing the autosave/share/export pipeline that already backs every `.clproj`.

Block mode (Scratch-style scripted editor)

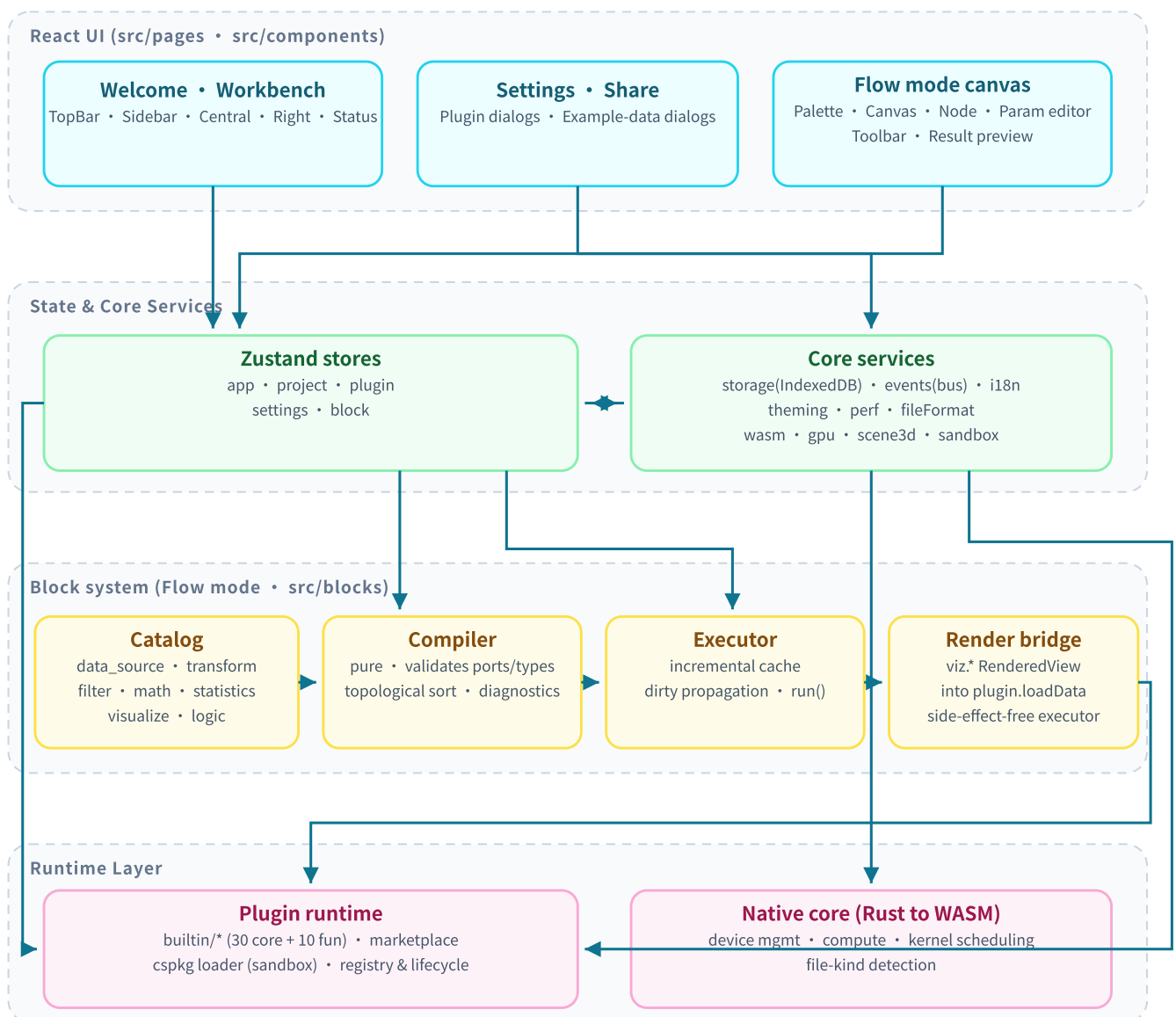
- A third workbench mode — `Standard | Flow | Blocks | Code` in the top bar. The Block mode is the entry point for learners and for anyone who wants an imperative feel: you write a top-down script of blocks under a single green 「运行时 / Run」 hat, and that hat is the only execution entry point (orphan blocks never run).
- 30+ built-in blocks organised by category — Start, Data, Variables, Operators, Transform, Statistics, Visualize, Control, Utility — covering data sources (`load CSV`, `load XYZ`, `random`, `range`), transforms (`normalize`, `sort`, `select`, `filter`), statistics (`summary`, `histogram`), plots (`scatter`, `line`, `histogram`, `point cloud`), control flow (`if`, `repeat`, `while`, `for_each`), and 1-to-1 utility primitives (`set`, `print`).
- **Shared IR** (`src/editor/ir/`) is the single source of truth. Block JSON ↔ IR round-trips in a pure, Node-testable module — the same IR that Code mode shares with Block mode for bidirectional sync.
- **IR interpreter** (`src/editor/runtime/interpreter.ts`) walks the IR directly and calls into the **same studio.* API** that Flow-mode blocks use (`studio.load` / `normalize` / `plot` / `print` / ...), so `studio.plot( 'scatter', df, { x, y } )` lands in the very same scatter plugin a Flow- mode `viz.scatter` block does.
- **IR → JS / Python codegen** (`src/editor/codegen/`) emits runnable code from the IR; the toolbar's "Python" / "JS" toggle shows the live codegen result for the current workspace.
- **Blockly 13** powers the canvas (`src/editor/blockly/`); the package is **lazy-loaded** so Standard / Flow first paint is unaffected (~828 KB on-demand chunk).
- **Block names, tooltips, dropdown options, and toolbox categories are localised** through Blockly's `BKY_*` key system; switching language re-creates the workspace with re-labelled blocks and is verified by a dedicated unit test (`tests/editor/block-i18n.test.ts`).
- **Sample programs** live in `src/editor/block/samples.ts` (5 built-in pipelines: galaxy scatter, telemetry line, random histogram, normalised scatter, repeat-print) and are loaded via the **Examples** dialog in the top bar — discoverable by any user, one click away.

Code mode (Monaco + Pyodide Python runtime)

- A fourth workbench mode — `Standard | Flow | Blocks | Code` in the top bar. Code mode is the escape hatch for real scripting: write free-form **Python** in a Monaco editor and run it on **CPython via a Pyodide Web Worker**.

- **Same `studio.*` API** as block mode (`load / random / range / normalize / sort / select / addColumn / filter / summary / histogram / plot / print / notify / getParam / setParam`), injected as a real importable Python module; `studio.plot(...)` renders through the shared plugin bridge.
- **REPL** input in the console panel evaluates single expressions without a full re-run; **stop** terminates and respawns the worker so runaway loops cannot hang the page.
- **9 sample programs** live as real files under `examples/code/*.py` (loaded via `import.meta.glob`, display metadata in `src/editor/code/samples.ts`) and load through the **Examples** dialog — from a one-liner scatter to an EDA pipeline, Monte-Carlo π , and signal smoothing.

Architecture



- **Host ↔ plugin contract**: every plugin implements a `Plugin` interface (init/destroy/activate/deactivate/render/updateParams/getParams/compute/

loadData/renderToScene) and receives a `PluginApi` for locale, status, perf reporting, notifications, file access, and project-scoped params.

- **Isolation boundary:** sandboxed plugins communicate exclusively through a typed RPC protocol (`src/core/sandbox.ts` + `src/core/plugin-worker.ts`).
- **WebGPU:** `src/core/gpu.ts` manages the adapter/device with a CPU fallback; the Rust core (`native/ergalics-core`) exposes `BindingDescriptor`, `ComputeKernel` (`compile/dispatch/compilation_info`) and `GpuDeviceManager` to JS via wasm-bindgen.

Tech Stack

Layer	Choice
UI	React 18, react-router-dom 7, Zustand 5
Language	TypeScript 5.7 (strict)
Build	Vite 6
3D	Three.js r185 (+ @types/three)
Native	Rust → wasm32-unknown-unknown, wasm-bindgen 0.2
GPU	WebGPU / WGSL via web-sys
Testing	Vitest (unit) + Playwright-core (E2E, headless Edge)
Docs	VitePress (separate <code>docs/</code> workspace)
Packaging	fflate (cspkg ZIP), lz-string (project compression)

Getting Started

Prerequisites

- **Node.js** ≥ 20 and npm
- **Rust toolchain** with the `wasm32-unknown-unknown` target and `wasm-bindgen-cli` (only needed to build the native core; the frontend degrades gracefully when the WASM module is absent)

Install

```
npm install
```

Run in development

```
npm run dev
```

The app opens at the Vite dev server URL. The welcome page runs hardware self-checks (WebGPU, WASM, IndexedDB) before entering the workbench.

Build

```
npm run build      # wasm → typecheck → vite build
npm run build:web   # frontend only (no WASM)
npm run build:wasm  # rebuild the Rust core into src/native
```

The production build is emitted to `dist/`. Note that `build:wasm` runs before `vite build` so the WASM bindings are always fresh.

Docs site

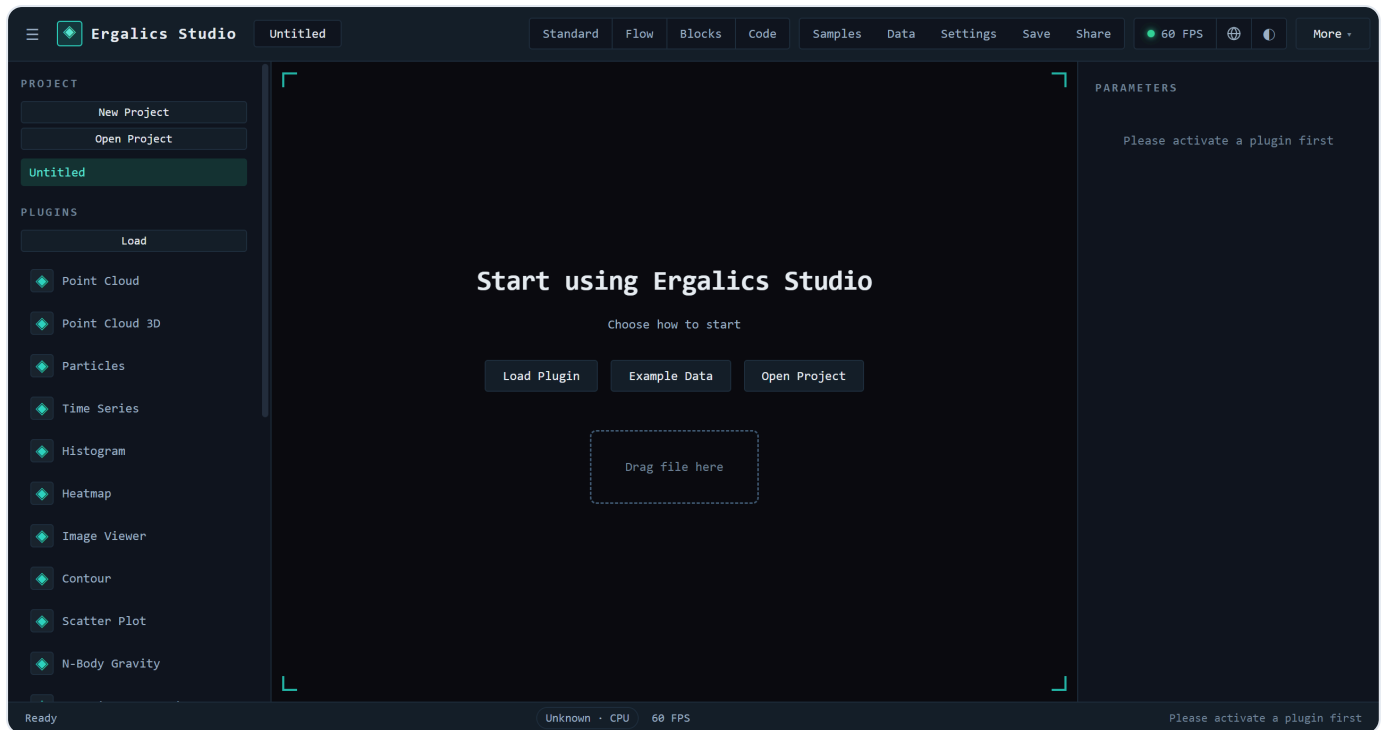
```
cd docs && npm install && npm run dev
```

See [Documentation](#) for details.

Project Layout

```
.
├─ src/                                # Frontend
│   ├─ core/                          # services: storage, events, i18n, gpu, wasm,
│   │                                     # fileFormat, scene3d, sandbox, cspkg,
│   │                                     # stats (statistics kernel), io (HDF5/NetCDF/
│   │                                     # FITS/Zarr/Parquet), plot (SVG/PDF engine),
│   │                                     # repro (reproducibility), logger, ...
│   └─ blocks/                        # block system (Flow mode):
│       │                             # types · registry · compiler · executor ·
│       │                             # ops · catalog · sample · l10n · render
│   └─ editor/                        # Block & Code modes:
│       │                             # ir · block (Blockly) · code · codegen ·
│       │                             # runtime (StudioApi + interpreter)
│   └─ components/blocks/            # Flow-mode canvas, palette, node, param editor,
│       │                             # toolbar, result preview, workbench shell
│   └─ components/editor/            # Block/Code canvases, variable / console panels
│   └─ pages/                        # welcome, workbench, settings, share, dialogs
│   └─ plugins/builtin/              # 27 core + 10 fun/utility plugins (2D + 3D)
│   └─ plugins/marketplace.ts        # marketplace catalog (tags/popularity/filters)
│   └─ stores/                      # zustand stores (app/project/plugin/settings/block/editor)
│   └─ types/                       # plugin & project & editor contracts
│   └─ native/                      # generated WASM bindings (git-untracked)
├─ native/ergalics-core/            # Rust core (device, compute, utils)
├─ examples/
│   ├─ data/                        # sample datasets used by the example plugins
│   └─ projects/                    # sample `.clproj` projects (incl. Flow pipelines)
│       └─ code/                    # sample Python programs for Code mode (*.py)
├─ scripts/                         # build-wasm · make-example-data · E2E suites
├─ tests/                           # Vitest unit tests
└─ docs/                            # VitePress documentation workspace
```

Standard Mode

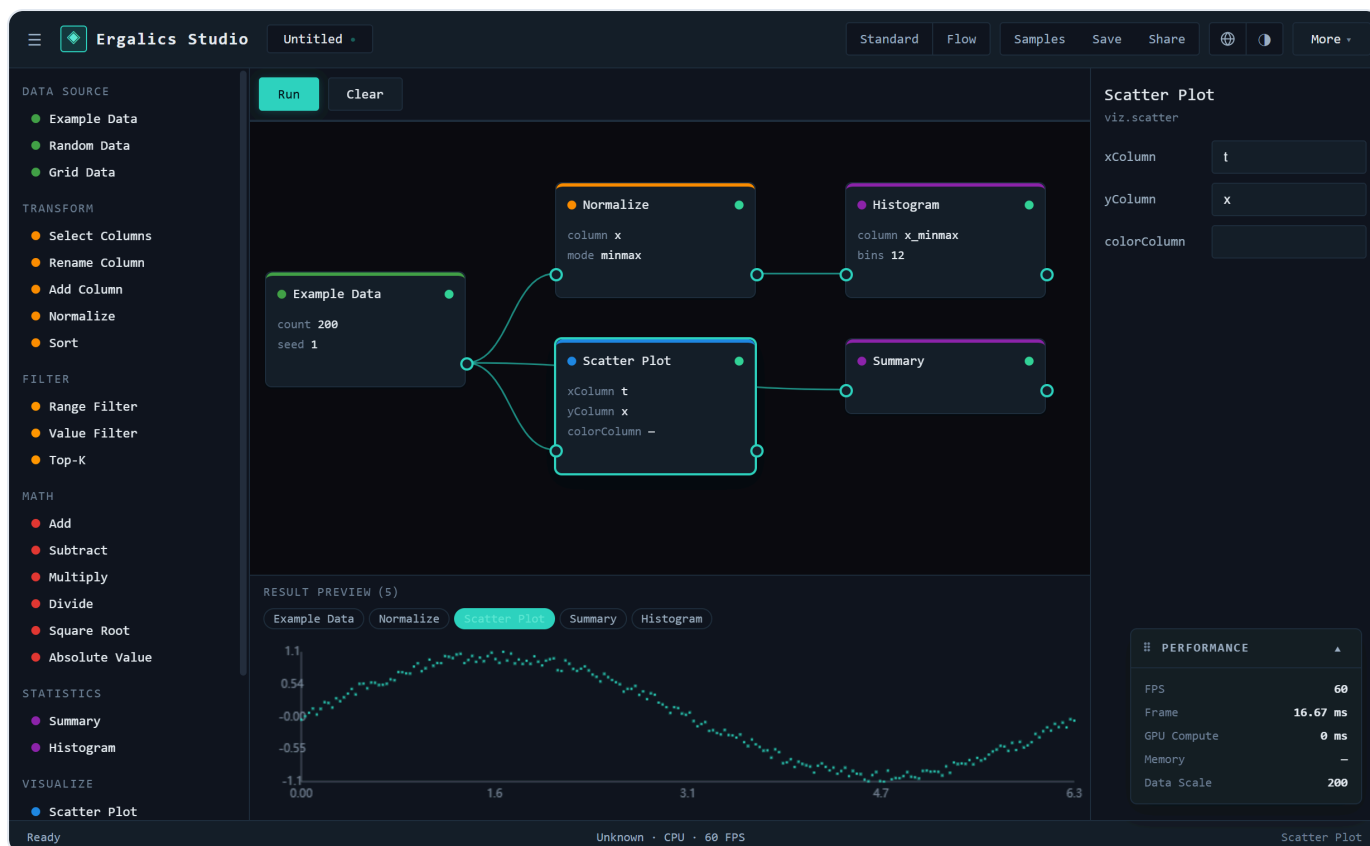


Standard mode — drag a file, see a visualisation

The default landing experience. Three panels: a **left rail** that lists your projects and plugins, a **centre viewport** that hosts whichever plugin is active (with a drop zone on first launch), and a **right panel** that turns the active plugin's declared parameters into reactive form fields. Files dragged onto the centre (or onto the plugin list) are routed by extension and magic number to a matching plugin; when more than one plugin matches, a chooser dialog lets you decide.

This is the mode you want when you already know which plugin answers your question and just need to point it at a file.

Flow Mode



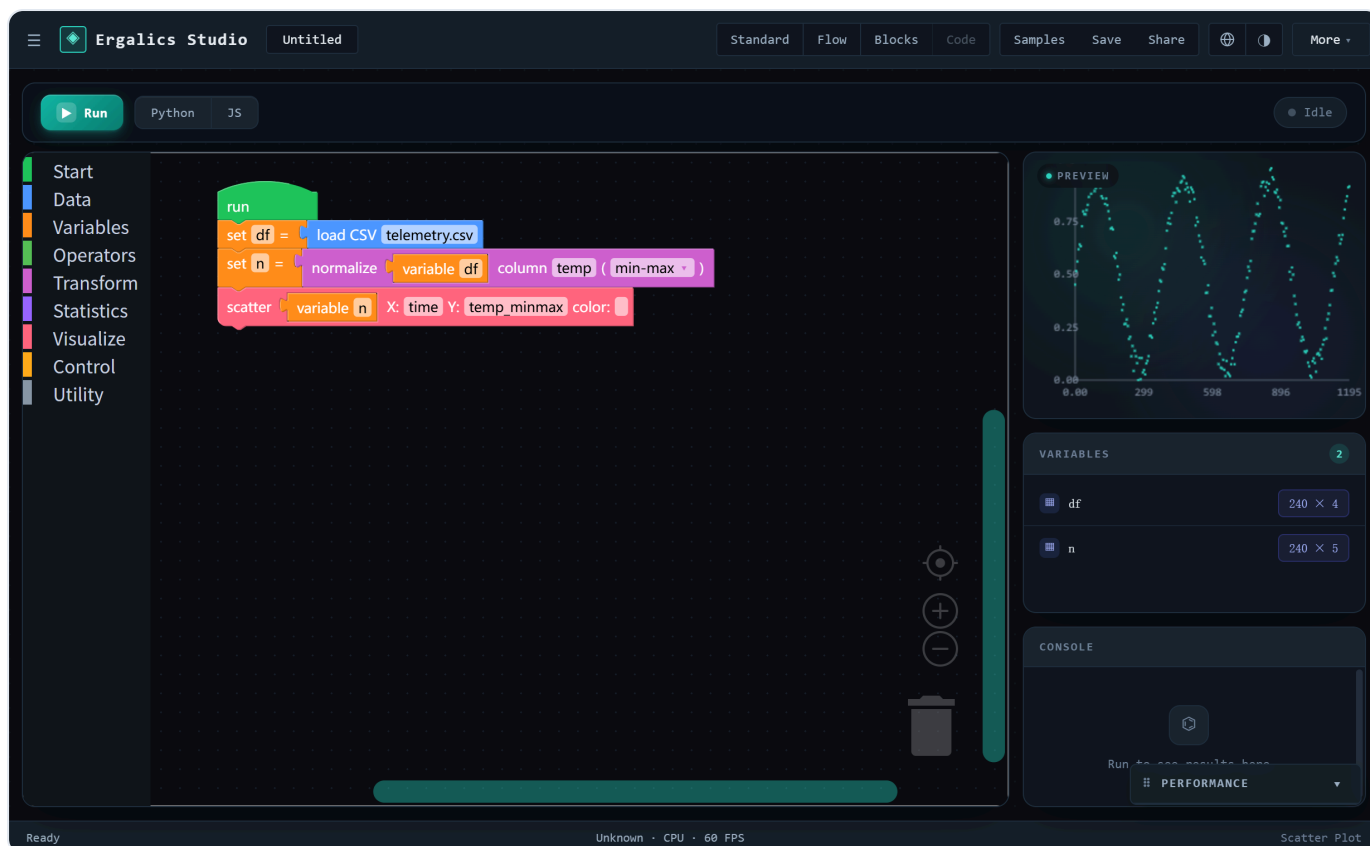
Flow mode — a sample pipeline (Normalize • Histogram / Scatter / Summary) with live result preview

A second workbench mode. Instead of *using* a plugin, you **compose a visual dataflow pipeline** from built-in blocks and run it. The pipeline editor lives on the left of the screen (palette), the canvas in the middle (nodes

- edges), a parameter editor on the right for whichever node is selected, and a live **result preview** at the bottom that adapts to the chosen node's output type:
- **RenderedView** (any `viz.*` node) — routed through the existing plugin renderer (scatter, histogram, ...).
- **DataTable** (a `stats.summary` / `stats.histogram` row) — a read-only table so non-viz outputs become visible too.
- **Scalar** — an inline value.

The graph persists into the project's `blockGraph` and rehydrates on open, sharing the autosave/share/export pipeline that backs every `.c1proj`. See <docs/guide/flow-mode.md> for the architecture (compiler + incremental executor + the render bridge), and the sample pipelines under `examples/projects/`.

Block Mode



Block mode — a "Run" hat block kicks off a program that loads telemetry, normalises, and plots

A Scratch-style block editor for fully scripted programs. A single green 「运行时 / Run」 hat block is the only entry point — anything not connected underneath it is ignored at run time, which makes "broken code" impossible to execute accidentally. Below the hat, blocks wire together into a top-down script: `set df = load CSV telemetry.csv` → `set n = normalize df column temp min-max` → `scatter df X:time Y:temp_minmax color:...`.

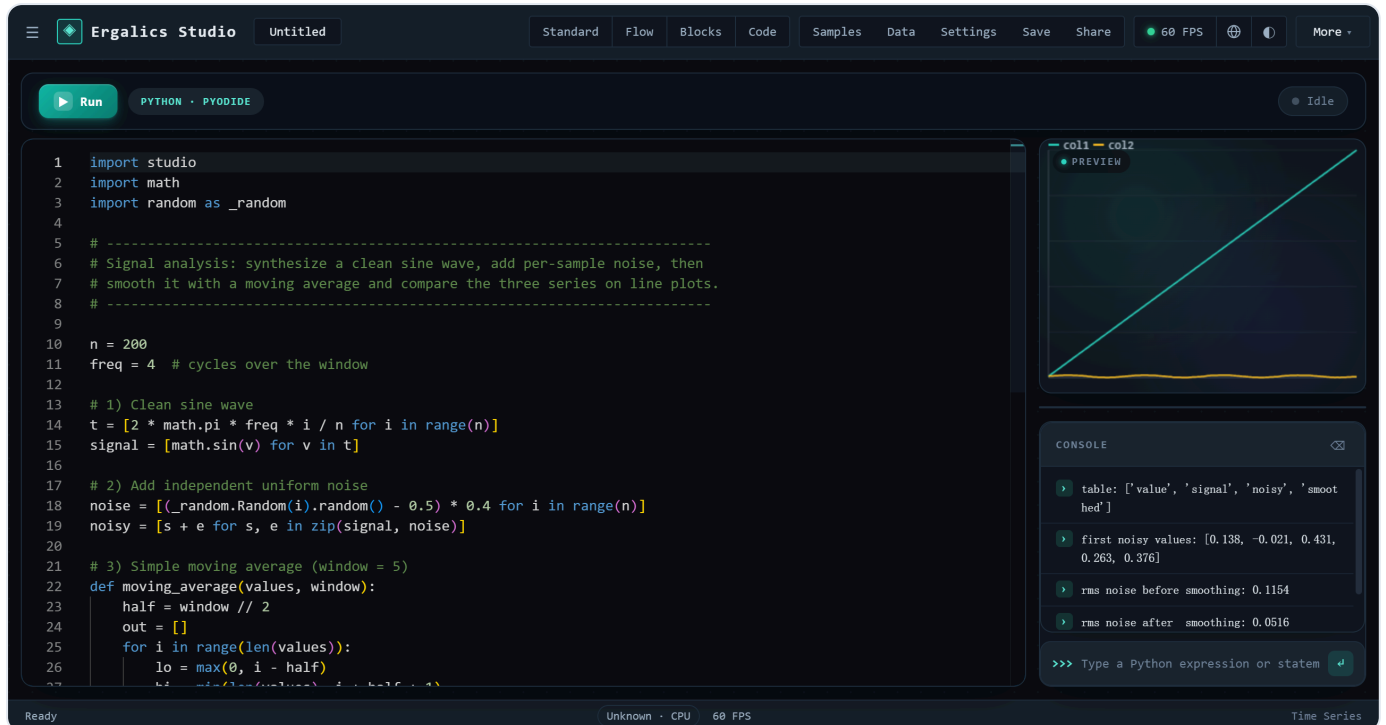
The run button ships a **live result preview**, **variables** panel, and **console** panel in the right-hand cards, so each run shows you what your data became and what got printed.

Under the hood:

- **Shared IR** (`src/editor/ir/`) is the single source of truth for both Block mode and Code mode. Block JSON ↔ IR round-trips in a Node-testable pure module.
- **IR interpreter** (`src/editor/runtime/interpreter.ts`) walks the IR directly and calls into the same `studio.*` API that the Flow mode blocks use — so `studio.plot('scatter', df, { x, y })` lands in the very same scatter plugin as a Flow-mode `viz.scatter` block.
- **IR → JS / Python codegen** (`src/editor/codegen/`) reuses the same IR to emit code, powering the "view code" overlay in the toolbar.
- **Blockly 13** (`src/editor/blockly/`) provides the canvas; the package is **lazy-loaded** so Standard/Flow first paint is unaffected (~828 KB chunk on demand).
- **i18n** is wired through `BKY_*` keys into Blockly's locale system — switching language re-creates the workspace with re-labelled blocks.

See [docs/guide/block-mode.md](#) for the full architecture, the 30+ built-in blocks, the 5 sample programs, and the limitations / next steps.

Code Mode



Code mode — a Monaco Python editor backed by a Pyodide worker, with a REPL console and a live plot preview

A real Python editor for the fourth workbench mode. Code mode runs **CPython in the browser** through a Pyodide Web Worker, so you write free-form Python against the same `studio.*` API that block mode generates — no scaffolding, no context switching.

- **Monaco editor** (`src/components/editor/CodeEditor.tsx`) with Python syntax highlighting, dark/light theming, word wrap, and `studio.*` autocompletion.
- **Pyodide worker runtime** (`src/core/pyodide/`) — real CPython in a Web Worker. The `studio` module is injected as a proper importable module (`sys.modules['studio']`), and project data files ship into the worker as `_FILES` so `studio.load('telemetry.csv')` resolves synchronously.
- **Same Studio API as block mode** — `studio.load / random / range / normalize / sort / select / addColumn / filter / summary / histogram / plot / print / notify / getParam / setParam`. `studio.plot(...)` renders through the exact same plugin bridge as a Flow-mode `viz.*` block, so a plot lands in the very same scatter / line / histogram plugin.
- **REPL** — evaluate a single Python expression or statement from the console input without re-running the whole program.
- **Interrupt** — stopping a run terminates and respawns the worker, so a runaway loop cannot hang the page.

- **9 sample programs** live as real files under `examples/code/*.py` (mirroring flow-mode's `examples/projects/`) and load through the **示例 / Examples** dialog — from one-liner scatters to a full EDA pipeline, a Monte-Carlo π estimation, and signal smoothing.

The IR shared with block mode (`src/editor/ir/`), the IR interpreter, and the IR $\rightarrow$ JS / Python codegen are all reused here, so block and code modes stay consistent on the same data semantics.

Three-mode conversion — the shared IR is the single hub for all three editing modes:

`src/editor/flow/convert.ts` round-trips IR $\leftrightarrow$ Flow DAG (`irToFlow` / `flowToIR`), and

`src/editor/block/convert.ts` round-trips Blockly JSON $\leftrightarrow$ IR (`blockJSONToIR` / `irToBlockJSON`).

`src/editor/code/parse.ts` additionally parses `studio.*` calls in a Code-mode buffer back into the IR (`parseCodeToIR`), preserving unparsed lines as raw-code nodes. Edit a pipeline in Flow mode, switch to Blocks and see the same logic as Scratch blocks, then jump to Code for the generated Python — all driven by one IR. A dedicated `sync-threeway` unit test pins the round-trip in both directions.

See `docs/guide/block-mode.md` for the architecture; R via webR is the remaining runtime.

Plugin System

Built-in plugins

Core / scientific plugins (auto-loaded at startup, 30 total):

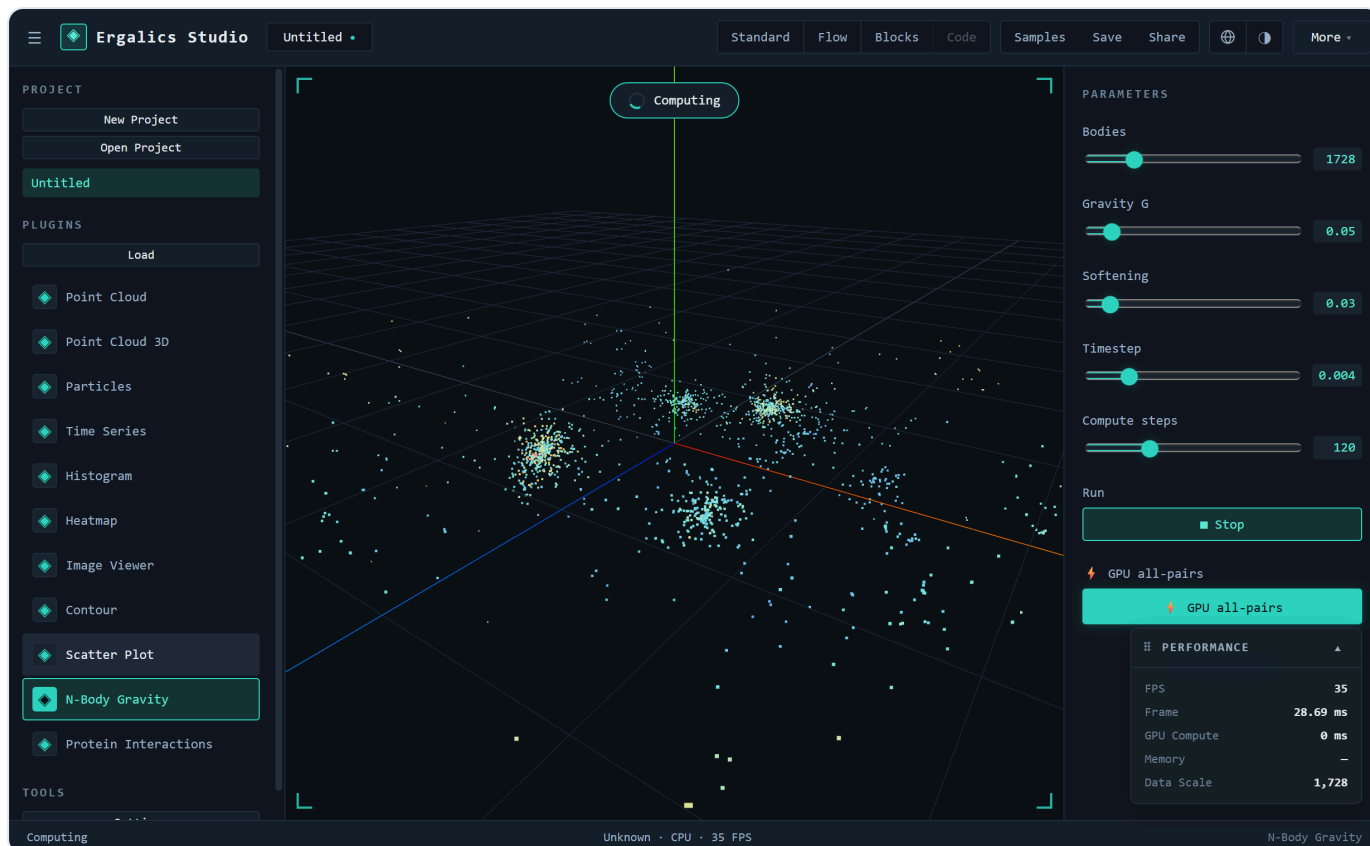
Plugin	Data	Capability
Point Cloud	<code>.xyz</code>	2D canvas
Point Cloud 3D	<code>.xyz</code> , <code>.dat</code>	Three.js scene, height ramp
Particles	<code>.dat</code>	2D simulation + real WGSL compute + progress
Time Series	<code>.csv</code>	2D line charts
Histogram	<code>.dat</code>	binning + log scale
Heatmap	<code>.json</code> (grid)	viridis ramp
Image Viewer	<code>.png</code>	base64 asset
Contour	<code>.json</code> (grid)	color ramp + isolines
Scatter	<code>.dat</code> , <code>.csv</code> , <code>.xyz</code>	2D scatter, color channel
N-Body Gravity	<code>.json</code> (bodies)	3D Three.js points + WGSL all-pairs gravity
Protein Interactions	<code>.json</code> (network)	force-directed layout + component metrics
Bar Chart	<code>.csv</code> (category, value)	grouped bars, orientation & palette
Polar / Radar Plot	<code>.csv</code> (dimension × series)	multi-series radar
Network Graph	<code>.csv</code> (source, target, weight)	force-directed layout, degree sizing
Bubble Chart	<code>.csv</code> (x, y, size, color)	bubble size + color channels
Violin Plot	<code>.csv</code> (group, value)	kernel density + box overlay
Sankey Diagram	<code>.csv</code> (source, target, value)	proportional flow ribbons
Box Plot	<code>.csv</code> (group, value)	quartiles, whiskers, outliers
Parallel Coordinates	<code>.csv</code> (multi-variate)	categorical coloring
Error Band	<code>.csv</code> (x, y, err)	shaded confidence band
Treemap	<code>.csv</code> (label, size / label, parent, size)	hierarchical rectangle layout
QQ Plot	<code>.csv</code> , <code>.dat</code> (single column)	normal quantile comparison + reference line
AI Trainer	<code>.csv</code> , <code>.json</code> (MNIST)	4 models (linear / non-linear NN / logistic / CNN) with live loss curve, scatter+fit / decision boundary / digit grid

Plugin	Data	Capability
LBM Fluid	<code>.json</code> (obstacle mask)	2-D lattice-Boltzmann channel flow (D2Q9) around an obstacle; WGSL collide + stream kernels, Kármán vortex street
Wave Equation	<code>.json</code> (u / drive grids)	2-D finite-difference wave equation (pulse / twin-source interference / double-slit scenarios); WGSL leapfrog kernel
Double Pendulum	<code>.json</code> (initial conditions)	RK4 integration with a chaos ghost twin offset by 0.001 rad — sensitive dependence made visible
GeoJSON Map	<code>.geojson</code> , <code>.json</code>	offline vector map with choropleth shading; Albers (China) / Web Mercator / equirectangular projections
Electromagnetism	<code>.json</code> (charges / fields)	draggable charges under Coulomb + Lorentz forces in a uniform B field; cyclotron spirals
Optics Lab	<code>.json</code> (optical layout)	geometric ray tracing with thin lenses, a Snell + dispersion prism, and a draggable light screen
Structural Mechanics	<code>.json</code> (truss members)	pin-jointed truss with axial-force coloring, utilization readouts and overload collapse

Simulation plugins are strictly data-driven: they start empty and never fabricate a default scene — the flow obstacle, the wave scenario, and the pendulum initial conditions all come from a bundled sample or a user file, and **Reset** replays the loaded data rather than restoring a built-in.

Every core plugin ships with a sample dataset (see `examples/data/`) so a one-click load in the **示例 / Examples** dialog produces a real visualisation immediately. Four of the plugins that lean on a non-trivial compute path are shown below — three WGSL compute demos and the TF.js-powered in-browser trainer:

N-Body Gravity — direct-summation gravity, $O(N^2)$ per step, on the GPU.



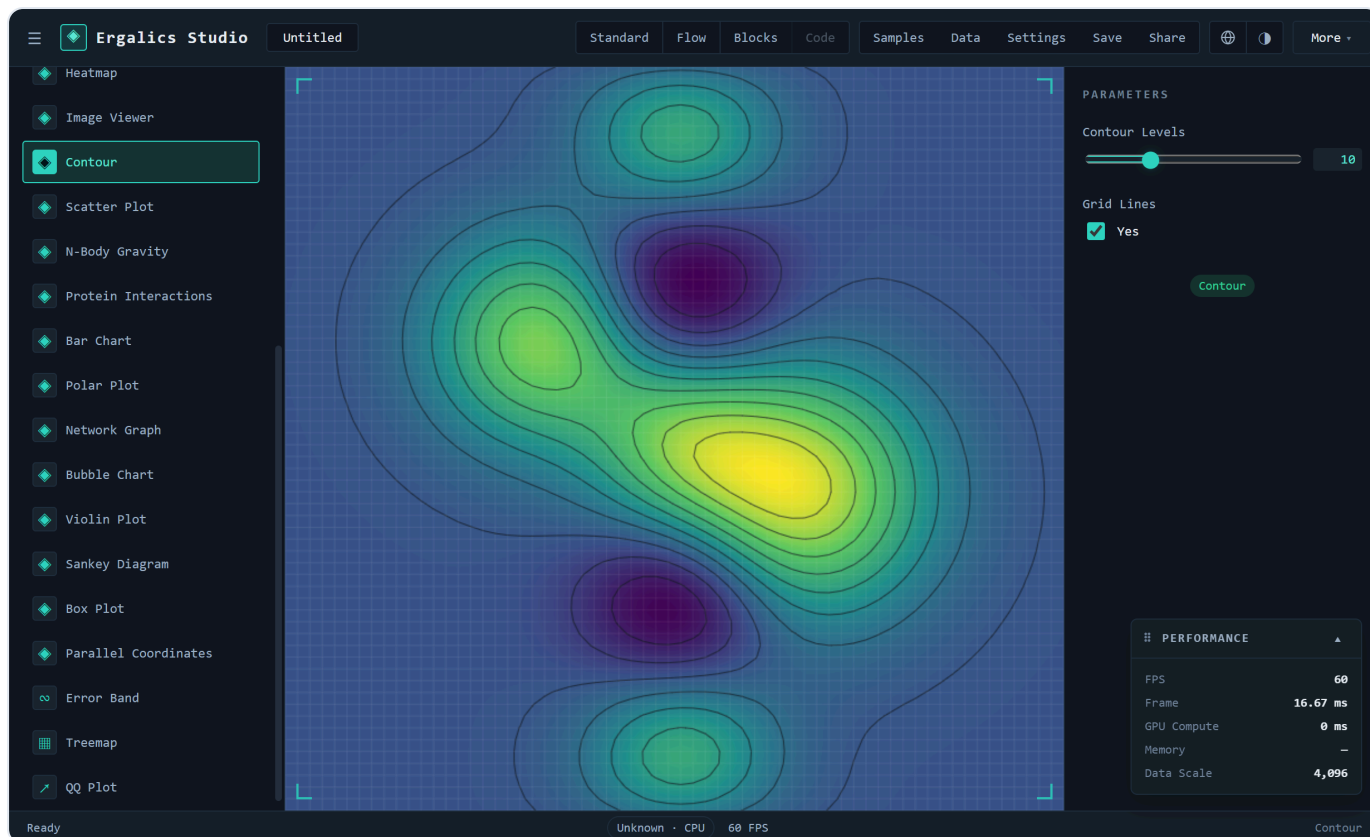
N-Body Gravity — a 4096-body torus ring orbiting a central mass (3D, WGSL all-pairs)

Protein Interactions — force-directed layout of a PPI network with component metrics.



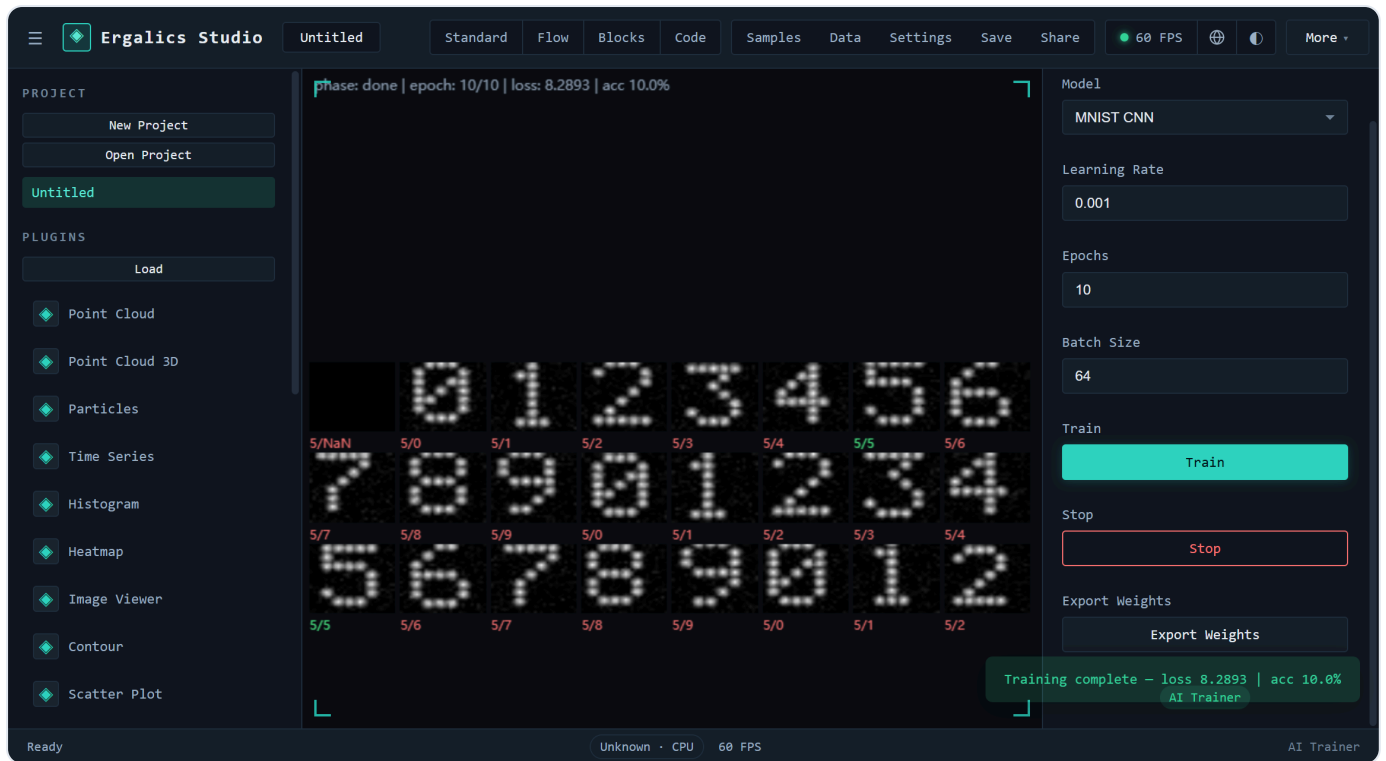
Protein Interactions — a force-directed layout of a 560-protein / ~1700-interaction network

Contour — a 64×64 twin-peak scalar field rendered with the viridis ramp and isolines.



Contour — twin gaussian peaks with wavy ridge, viridis ramp + isolines

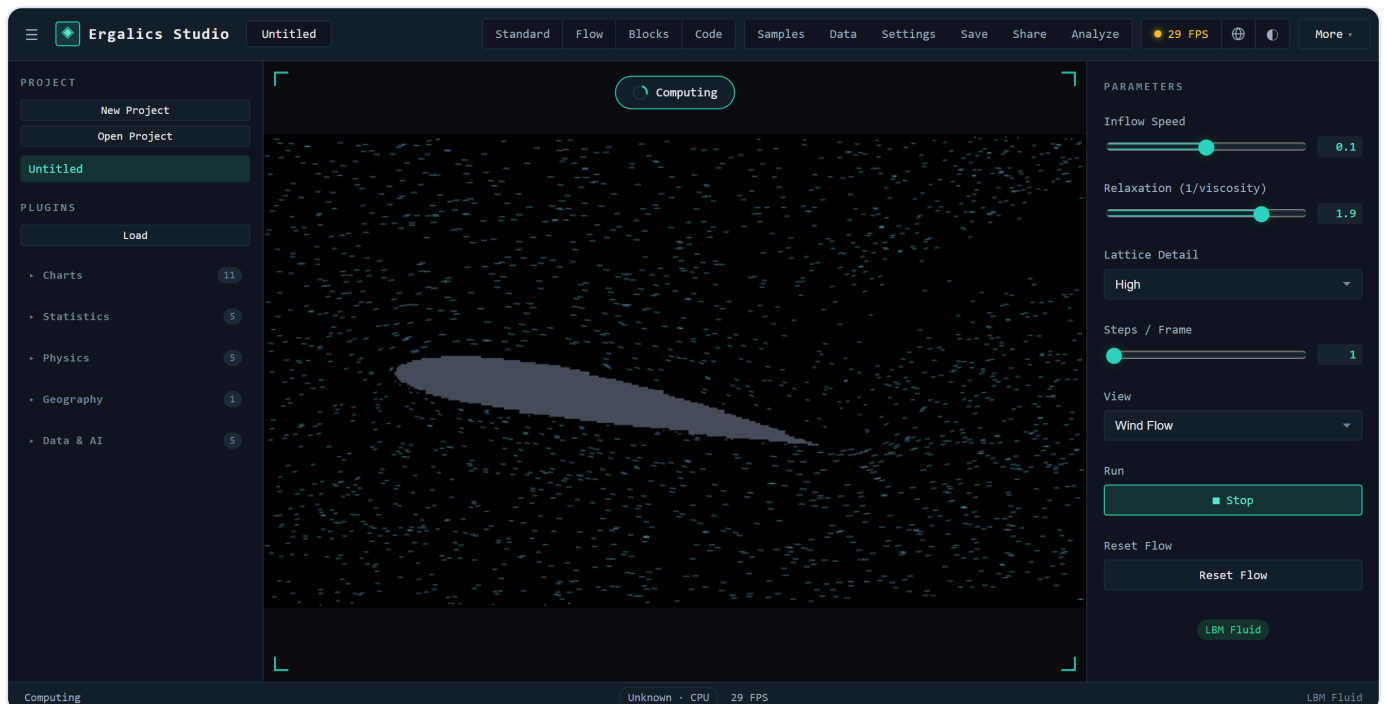
AI Trainer — train linear / non-linear / logistic / convolutional models in the browser with TensorFlow.js. The top of the canvas shows the live loss curve, and the lower panel switches between scatter+fit, a 2-D decision boundary, and a digit grid depending on the model. Four bundled samples (`examples/data/ai/*.csv`) cover a linear regression, a cubic+sine curve, a two-gaussian classification, and a 200-image MNIST subset. The TF.js bundle itself is lazy-loaded on the first click of **Train**, so the trainer is in the auto-load registry without paying a 2 MB cost at startup.



AI Trainer — MNIST CNN trained for 10 epochs on a 200-image synthetic digit set, grid shows predictions (green) vs. ground truth (red)

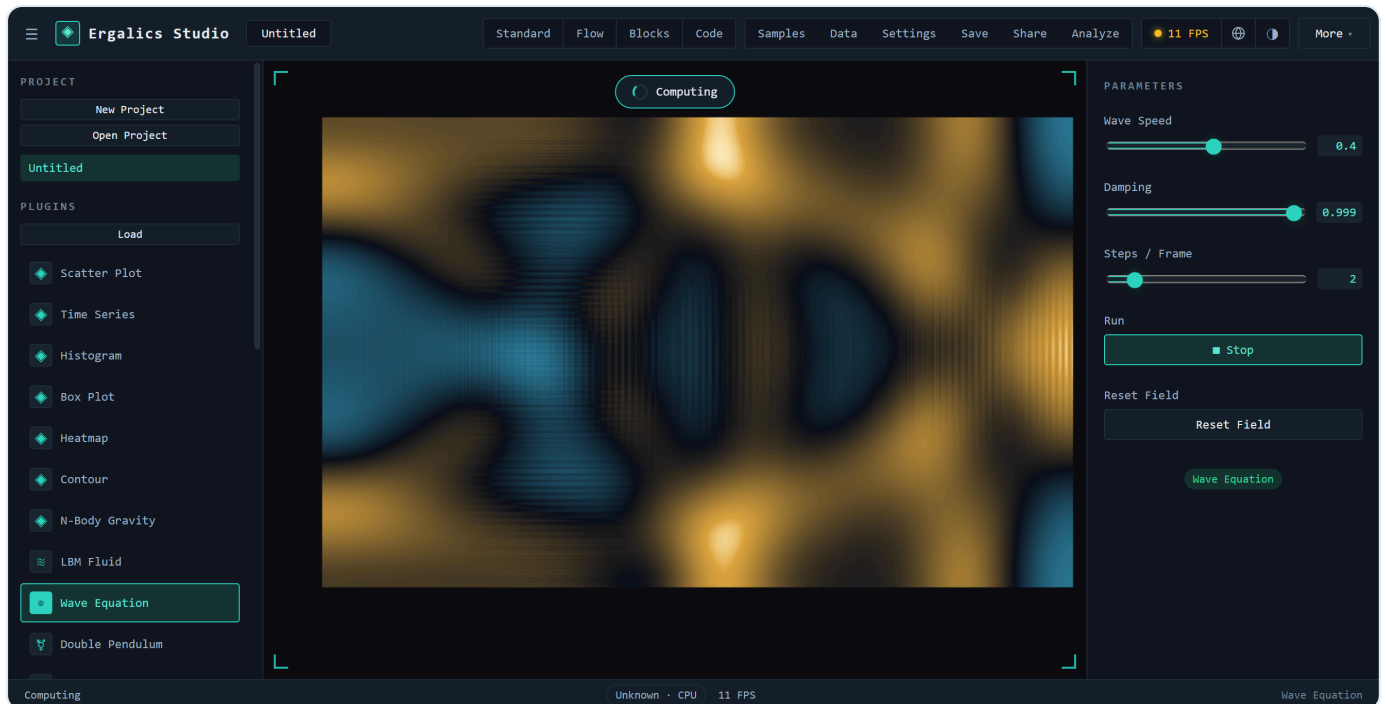
The simulation & geospatial plugins complete the registry — four more data-driven engines shown below:

LBM Fluid — a 2-D lattice-Boltzmann channel flow (D2Q9) around an obstacle, with WGS� collide + stream kernels. The bundled airfoil sample shows the flow field in the Wind Flow view; inflow speed, relaxation and lattice detail are live parameters.



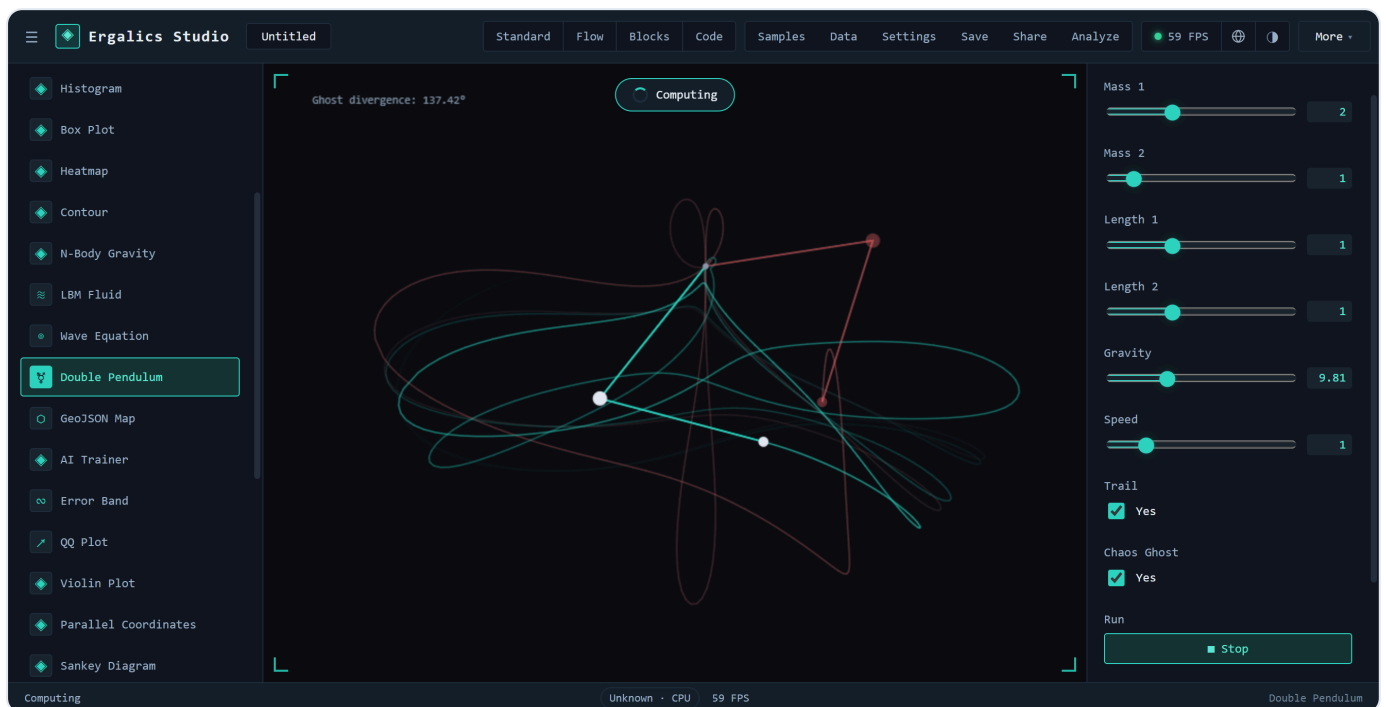
LBM Fluid — lattice-Boltzmann flow past an airfoil obstacle, Wind Flow view

Wave Equation — a 2-D finite-difference wave equation with pulse / twin-source interference / double-slit scenarios, integrated by a WGS� leapfrog kernel with adjustable wave speed and damping.



Wave Equation — twin-source interference pattern, orange/blue amplitude field

Double Pendulum — RK4 integration with a chaos ghost twin whose initial angle differs by 0.001 rad; the HUD reads out the live ghost divergence (137.42° in the frame below) as the two trajectories peel apart.



Double Pendulum — two trajectories diverging, HUD reads Ghost divergence: 137.42°

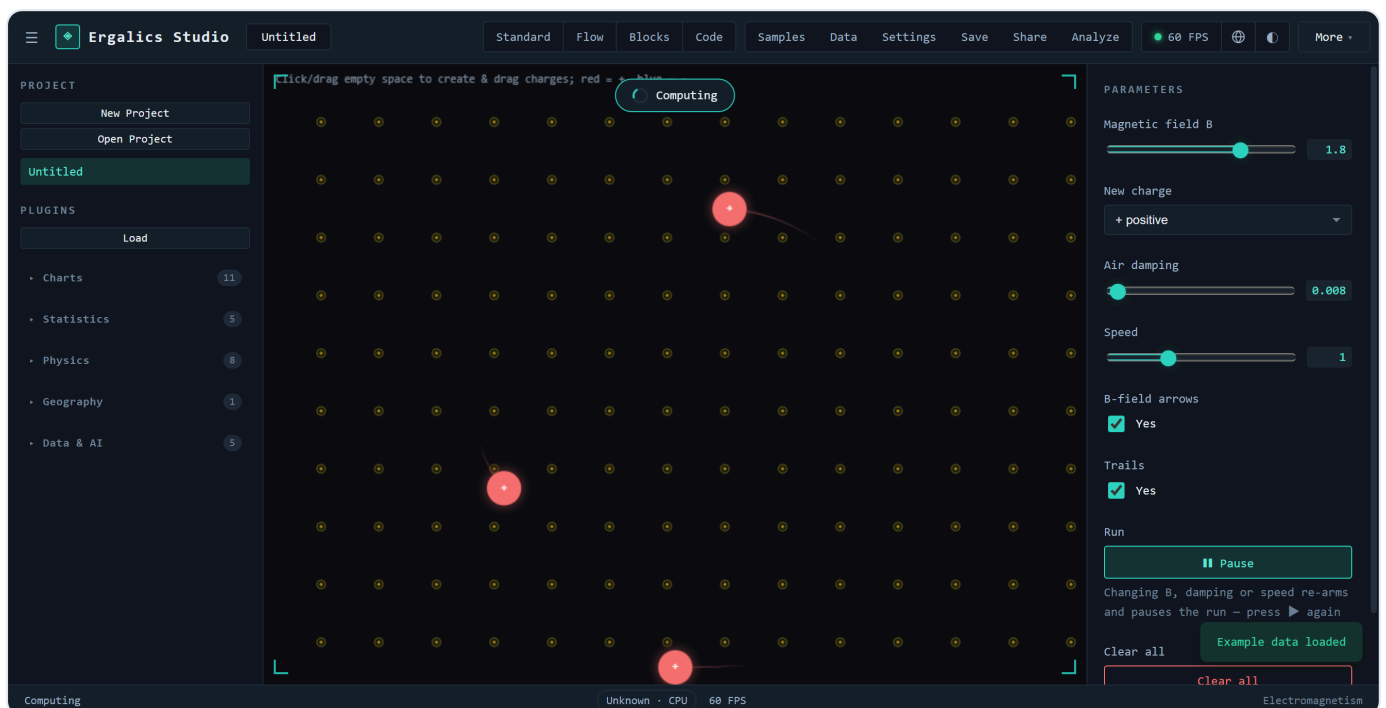
GeoJSON Map — an offline vector map with choropleth shading by property (`adcode` for the bundled China provinces sample), graticule, and Albers (China) / Web Mercator / equirectangular projections.



GeoJSON Map — China provinces choropleth in the Albers (China) projection

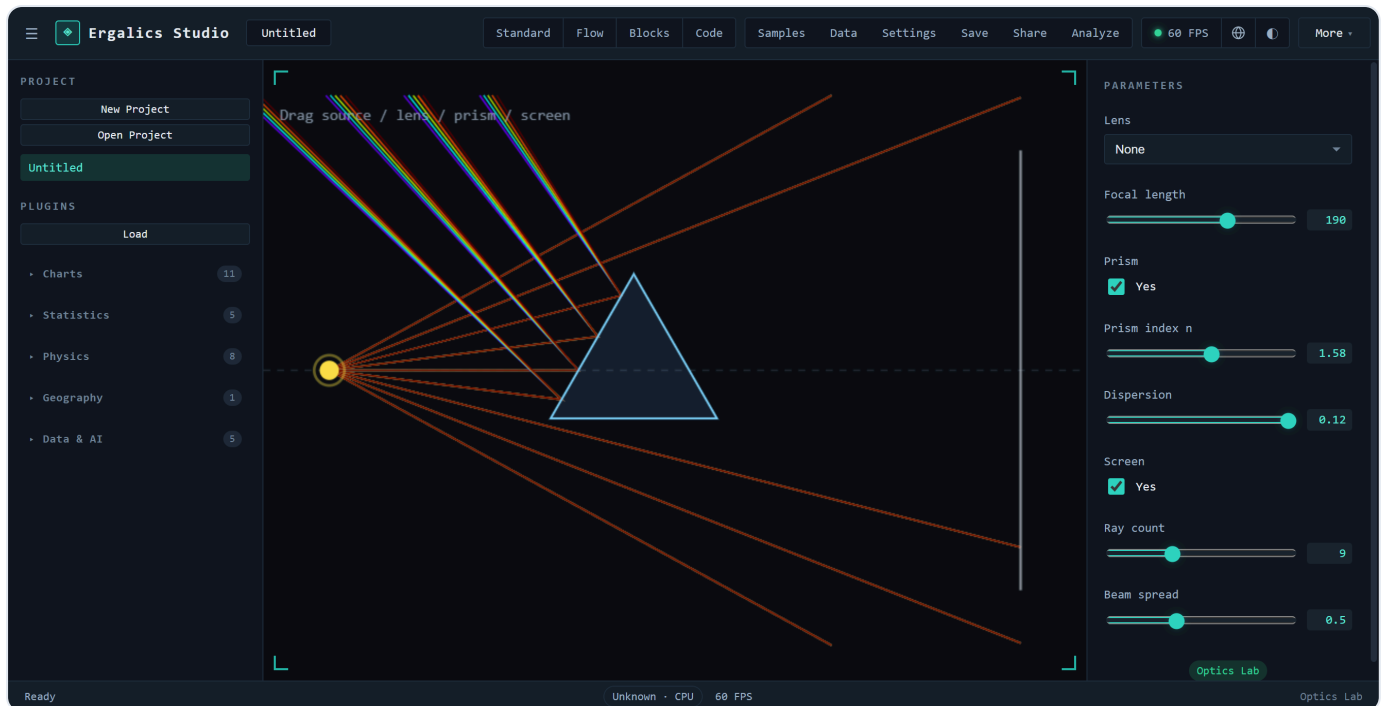
Three interactive physics labs round out the scientific registry — the flagship hands-on demos where objects are manipulated directly on the canvas:

Electromagnetism — draggable charges under Coulomb + Lorentz forces in a uniform B field (strength and direction are independent parameters). The cyclotron sample sends same-sign charges into spirals whose radius is set by velocity, mass and B.



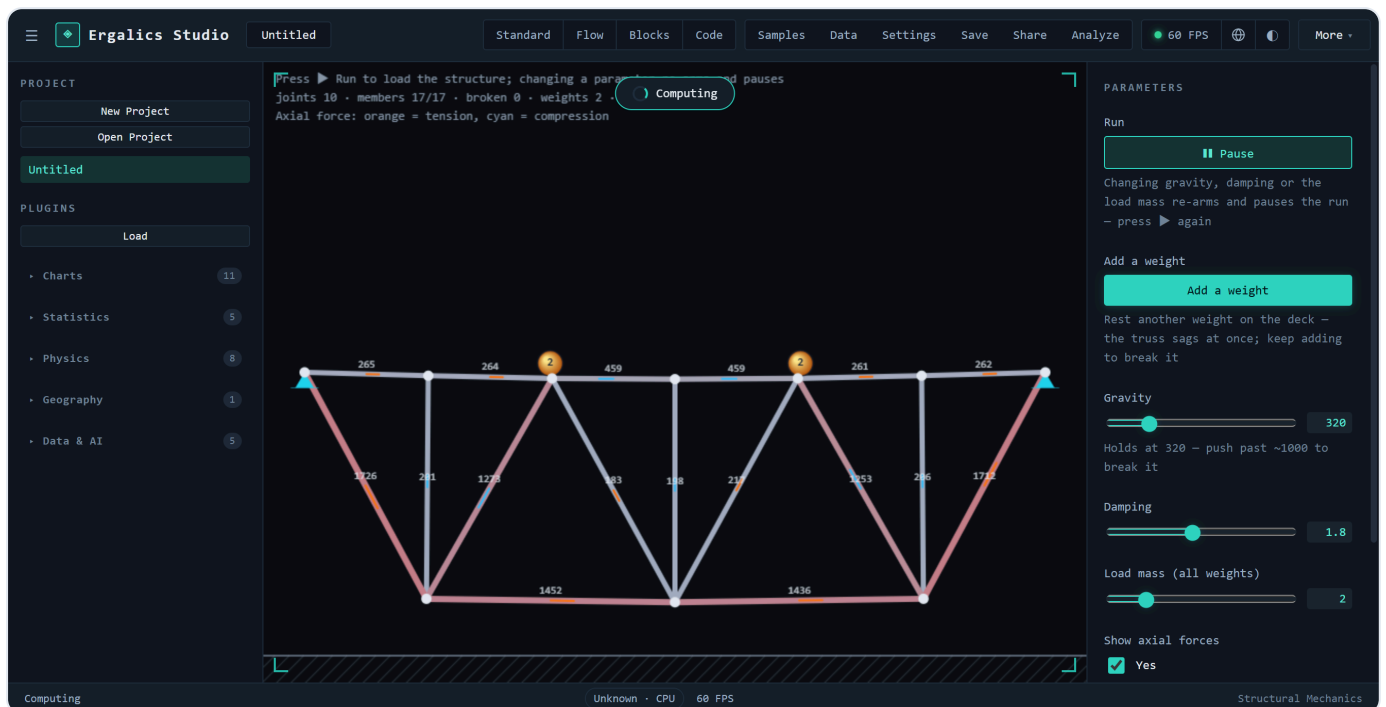
Electromagnetism — three positive charges spiraling in a uniform magnetic field (dot lattice = out-of-screen B), trails enabled

Optics Lab — geometric ray tracing with thin convex/concave lenses, a triangular prism with Snell refraction and dispersion, and a light screen; every element can be dragged on the canvas.



Optics Lab — a white beam dispersing through a triangular prism into a spectrum, collected on the screen

Structural Mechanics — a pin-jointed truss that carries its load live: members are colored by axial force (orange = tension, cyan = compression) with utilization readouts, and snap once overloaded until the frame collapses.



Structural Mechanics — a 17-member deck truss carrying two weights, members colored by axial force (orange = tension, cyan = compression)

Fun & utility plugins (`autoload: false` , 10 total — loaded on demand from the built-in panel or marketplace tab):

Plugin	Type	Description
Mandelbrot	fractal	Mandelbrot / Julia set browser with palettes and zoom
Spirograph	art	hypotrochoid curve art
Lissajous	art	animated Lissajous curves
Game of Life	toy	classic cellular automaton (play / pause / reseed)
Harmonograph	art	curve art from summed decaying sinusoids
Palette Explorer	utility	two-stop gradient preview + swatches
Koch Snowflake	fractal	recursive segment fractal
Barnsley Fern	fractal	iterated function system fern
Fireworks	toy	particle fireworks with gravity and trails
Truchet Tiles	pattern	random quarter-circle arc tiles

Every built-in runs the same math on CPU when WebGPU is absent — see [GPU Compute & Native Core](#) and [docs/guide/plugins.md](#) for the full list and their compute paths.

Third-party packages (`.cspkg`)

A package is a ZIP containing `manifest.json` plus the entry module and any assets. Loading validates the manifest (required fields, plugin-id format, entry path traversal, sandbox enum) and then executes the entry **inside a Web Worker sandbox** by default:

```
{
  "id": "com.example.analyzer",
  "name": "Analyzer",
  "version": "1.2.0",
  "author": "Example Corp",
  "description": "...",
  "entry": "dist/index.js",
  "sandbox": "isolated",          // "isolated" (default) | "trusted"
  "formats": [{ "extension": ".dat" }]
}
```

- `sandbox: "isolated"` (default) — runs in a Worker: separate global scope, no DOM/window/store access; canvas rendering via `OffscreenCanvas`.
- `sandbox: "trusted"` — executes in the host context with full DOM access. Only use for packages you control.

Limitations (documented honestly): workers share the origin's IndexedDB, and the legacy fallback (`new Function` with shadowed globals) is a best-effort approximation, **not** a security boundary. The UI warns

when the fallback is used.

GPU Compute & Native Core

The Rust crate `native/ergalics-core` compiles to `wasm32-unknown-unknown` and is bound with `wasm-bindgen`. Current surface:

- `GpuDeviceManager` — adapter/device acquisition with CPU-fallback option.
- `GpuBuffer` — the missing buffer half of the compute foundation: create with an explicit usage mask (`create_storage`, `create_readable_storage`, `create_uniform`), upload bytes with `write`, and read results back with `read` (copies into a dedicated `MAP_READ | COPY_DST` readback buffer — see the buffer-usage note below).
- `KernelDescriptor` + `BindingDescriptor` — describe a compute kernel and its buffer bindings (uniform / storage / read-only-storage, dynamic offsets, min binding size).
- `ComputeKernel::compile` — builds a **real** `GPUBindGroupLayout` from the binding descriptors, compiles the WGSL module, and creates the pipeline.
- `ComputeKernel::bind_group` — materializes a bind group (buffer $i \rightarrow$ binding i) from the kernel's retained layout.
- `ComputeKernel::run(queue, buffers, x, y, z)` — bind group + dispatch + submit in one call; `dispatch(queue, bindGroup, x, y, z)` remains for host-managed command encoders.
- `ComputeKernel::compilation_info()` — surfaces WGSL compile diagnostics (error/warning + line/column) asynchronously.
- `detect_file_kind` — magic-number file detection used by the loader.

Host-side compute service

`src/core/gpu.ts` owns the adapter/device lifecycle (CPU fallback, OOM tracking). On top of it, `src/core/compute.ts` exposes the **plugin-facing compute surface** (`PluginApi.gpu`): `createBuffer` / `write` / `read`, `compileKernel` + `compilationInfo`, and one-shot `run`. It routes through the Rust core when the WASM module is loaded and through the raw WebGPU API otherwise — so accelerated compute works in dev *and* production, and the Rust core stays the reference engine.

Reusable WGSL kernels live in `src/core/wgsl.ts` (particle integration, 3-D all-pairs N-body gravity, D2Q9 lattice-Boltzmann collide/stream/curl, and the 2-D wave-equation leapfrog), paired with host-side pack/unpack helpers that mirror the kernel math for the CPU fallback. The Particles plugin demonstrates the single-buffer path (upload interleaved `[x, y, vx, vy]` + uniform params $\rightarrow$ dispatch the WGSL integrator $\rightarrow$ read back $\rightarrow$ report real GPU time); the N-Body plugin demonstrates the heavier all-pairs path with ping-pong buffers that keep every integration step on the device with no per-step read-back.

When WebGPU (or the WASM module) is unavailable, `api.gpu` is `undefined` and plugins fall back to CPU — same behaviour, no GPU required.

Testing

Unit tests (Vitest, node environment):

```
npm test          # or npm run test:unit
npm run verify    # typecheck + unit tests
```

417 tests across 46 test files: file-format detection, scientific binary I/O (NetCDF/HDF5/FITS/Parquet/Zarr helpers), the statistics kernel (descriptive, special functions, tests, effect sizes, corrections, power), cspkg parsing/validation, sandbox RPC (including an end-to-end round trip through a fake Worker), i18n, app store, WASM retry policy, GPU compute (WGSL templates — particles, N-Body, histogram, heatmap, point-cloud — buffer packing, CPU integrators, service gating), built-in plugin logic, the data plugins' parsing helpers (error-band rows, treemap hierarchy, QQ probit), the block system end-to-end — `DataTable` ops, registry, compiler (validation/topology/type-check), executor (incremental cache + invalidation), geometry, catalog executors, the `viz.*` → plugin render bridge, codegen (JS/Python), three-mode IR sync (block ↔ flow ↔ code), the Pyodide worker protocol, the structural-mechanics simulator, plugin runtime lifecycle and recent bugfix regressions, the publication-grade plot engine, the reproducibility kernel, and the pipeline samples that load via `import.meta.glob`.

E2E suites (Playwright-core, headless Edge) against a production preview:

```
npm run test:e2e
```

Suite	Covers
<code>smoke-test</code>	boot, auto-loaded plugins, reactive params, project restore
<code>verify-ui</code>	layout, theming, canvas, plugin list
<code>verify-fixes</code>	all example plugins render their sample data correctly
<code>verify-3d</code>	3D point cloud in the host Three.js scene
<code>verify-plugins</code>	3D↔2D surface visibility, contour, scatter, tornado sample
<code>verify-webgpu</code>	GPU compute kernels (histogram / heatmap / point cloud) + CPU fallback
<code>verify-block-mode</code>	block editor: mode switch, compile, run, block → code sync
<code>verify-code-mode</code>	Monaco + Pyodide: run a Python program, console, variables, plot
<code>verify-ai-samples</code>	AI Training: load all 4 samples (linear / non-linear / logistic / MNIST)
<code>verify-ai-training</code>	AI Trainer: activate, TF.js train, loss curve, model-switch reset, decision boundary, MNIST CNN grid

Documentation

A dedicated VitePress documentation workspace lives in `docs/`:

```
cd docs
npm install
npm run dev      # local documentation site
npm run build    # static site → docs/.vitepress/dist
```

The production frontend build copies the docs site into `dist/docs/`, so the welcome page's **Docs** link works from a preview server. The docs site can also be deployed independently (e.g. GitHub Pages).

Roadmap

See `docs/guide/roadmap.md` for the current status table. Highlights:

- ✓ Workbench layout, project management, file routing
- ✓ 40 built-in plugins (30 core + 10 fun/utility), cspkg loading, Worker sandbox
- ✓ Plugin marketplace catalog (curated tags / popularity / category filters, on-demand loading)
- ✓ WebGPU device management + real compute-kernel pipeline
- ✓ i18n, theming, perf monitoring, share links
- ✓ Flow mode — visual dataflow pipeline (compiler + incremental executor + 37 built-in blocks + canvas UI + sample pipelines in `examples/projects/`)
- ✓ Vitest unit tests + Playwright E2E suites
- ✓ Plugin compute surface (`api.gpu`), WGSL templates, Particles accelerated
- ✓ GPU acceleration across all example plugins (histogram/heatmap/point cloud)
- ☐ Plugin marketplace: package signing & third-party install pipeline
- ✓ GitHub Actions CI (unit + E2E + Pages deploy)
- ✓ Block mode (Scratch-like, Google Blockly) — see [Block Mode](#). 30+ built-in blocks, shared IR with the interpreter, lazy-loaded Blockly 13, and 5 sample programs; lives behind the `Blocks` top-bar slot.
- ✓ Code mode (Python via Pyodide) — Monaco editor, CPython worker runtime with a real importable `studio` module, REPL + variables, worker interrupt, and 9 sample programs under `examples/code/`; same IR shared with block mode.
- ✓ Three-mode conversion — Block ↔ Flow ↔ Code round-trip through the shared IR (`src/editor/flow/convert.ts` + `src/editor/block/convert.ts`), pinned by a `sync-threeway` unit test; Code-mode buffers parse back into the IR via `src/editor/code/parse.ts`
- ✓ Statistics subsystem — hypothesis tests, effect sizes, multiple-comparison corrections, power analysis (`src/core/stats/`), surfaced as 11 Flow-mode `stats.*` blocks
- ✓ Scientific binary I/O — HDF5 / NetCDF / FITS / Zarr / Parquet import via a single dispatcher (`src/core/io/`)

- ✓ Publication-grade plot engine with SVG/PDF export and a reproducibility kernel (`src/core/plot/` , `src/core/repro/`)
 - ☐ Code mode: R runtime (webR)
-

Contributing

1. Fork the repository and create a feature branch.
2. Keep changes small and covered by tests — `npm run verify` must stay green, and new plugin/feature work should ship with an E2E check.
3. Run `npm run test:e2e` before opening a pull request (requires the Edge browser at its default install path; adjust `EDGE` in the scripts otherwise).
4. Regenerate WASM bindings with `npm run build:wasm` when touching `native/ergalics-core`.

Report bugs and feature requests via [GitHub Issues](#).

License

MIT © 2026 [SnowLeopard-io](#)